# Process Documentation — Actions Performed

### 🛠️ First Installed pnpm - Used `npm` to install the pnpm executable.

- Fonts - Added `IBM Plex Sans` via Next.js `next/font/google`.
- File changed: `app/layout.tsx` — added `IBM_Plex_Sans` import and configured the font variable `--font-ibm-plex` on the `<body>` class.
- Colors - Defined a custom color palette in `app/globals.css` and created utility classes for text colors.
- Favicon - Created a simple SVG favicon and updated the app metadata to reference it.

## 🗺️ Route Groups in NextJS

In the `app` directory, nested folders are normally mapped to URL paths. However, you can mark a folder as a Route Group to prevent the folder from being included in the route's URL path.

This allows you to organize your files better into logical groups without affecting the URL path structure.

### 📐 Convention

To create a Route Group, simply wrap the folder name in parentheses. For example, a folder named `(admin)` would be treated as a Route Group.

### 🗂️ Organize routes without affecting the URL path

To organize routes without affecting the URL, create a group to keep related routes together. The folders in parenthesis will be omitted from the URL (e.g. `(marketing)` or `(shop)`)

### 🧩 How we'll create our folder

We're gonna create a `(auth)` folder to `app` folder. then gonna create a `login` folder to contain all login-related pages like login and forgot-password. Then gonna add a `page.tsx` file inside that `login` folder.

```
export default function LoginPage() {
    return(
        <div>
            <h1>A <b>production-ready course platform</b> built with <b>Next.js
15</b>, featuring secure authentication, payments, analytics, progress
tracking, drag-and-drop course creation, and a modern admin & customer
dashboard experience.</h1>
        </div>
    )
}
```

now go to route `http://localhost:3000/login` to see the result.

## ✨ shadcn/ui Addition (what we ran and what was created)

What was run

- Executed the shadcn CLI to add the `button` component:

```
pnpm dlx shadcn@latest add button
```

What the CLI produced (key files created or updated)

- `components.json` — configuration for the shadcn UI generator (style: `new-york`, RSC/TSX flags, aliases pointing `ui` → `@/components/ui`).
- `components/ui/button.tsx` — the generated Button component (exported `Button` and `buttonVariants`).
- `lib/utils.ts` — small utility helper `cn()` for merging class names (used by generated components).

Notes

- The project already uses Tailwind; shadcn was configured to use `app/globals.css` for its CSS variables.
- You can now import and use the generated `Button` in app code, e.g.:

```
import { Button } from "@/components/ui/button"

export default function Demo() {
    return <Button>Click me</Button>
}
```

- Add more components via the shadcn CLI

```
pnpm dlx shadcn@latest add alert-dialog avatar badge breadcrumb button card
chart checkbox collapsible dialog drawer dropdown-menu form input-otp input
label progress select separator sheet sidebar skeleton sonner table tabs
textarea toggle-group toggle tooltip
```

## Login Page UI Creation

Created a simple login page inside the `(auth)/login/page.tsx` file with the following content:

```
import { Button } from "@/components/ui/button";
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from
"@/components/ui/card";
import { SiGithub as GithubIcon, SiFacebook } from 'react-icons/si';
```

```
import { SiGoogle } from 'react-icons/si'

export default function LoginPage() {
    return(
        <Card>
            <CardHeader>
                <CardTitle className="text-xl">Welcome Back!</CardTitle>
                <CardDescription>Please Log In to Your
Account</CardDescription>
            </CardHeader>

            <CardContent className="flex flex-col gap-4">
                <Button variant="google" className="w-full">
                    <SiGoogle className="size-4"/>
                    Log In With Google
                </Button>
                <Button variant="facebook" className="w-full">
                    <SiFacebook className="size-4"/>
                    Log In With Facebook
                </Button>
                <Button variant="default" className="w-full">
                    <GithubIcon className="size-4"/>
                    Log In With Github
                </Button>
            </CardContent>
        </Card>
    )
}
```

Also add some codes in `app/auth/layout.tsx` to center the login card.

```
export default function AuthLayout({ children }: { children: ReactNode }) {
    return(
        <div className="relative flex min-h-svh flex-col items-center justify-
center ">
            <div className="flex w-full max-w-sm flex-col gap-6">
                {children}
            </div>
        </div>
    )
}
```

## Dark Theme Addition

To add dark theme support to the application, we made the following changes:

1. Open Shadcn UI's website and navigate to the "Docs" section.
2. Locate the "Dark Theme" documentation page. Open Next.js & Copy the provided code snippet for dark theme support. Create a new file named `theme-provider.tsx` inside the `components/ui`

folder and paste the copied code into this file.

```
"use client"

import * as React from "react"
import { ThemeProvider as NextThemesProvider } from "next-themes"

export function ThemeProvider({
  children,
  ...props
}: React.ComponentProps<typeof NextThemesProvider>) {
  return <NextThemesProvider {...props}>{children}</NextThemesProvider>
}
```

> This code have a "use client" directive at the top, indicating that it is a client-side component & it will get a JavaScript bundle shipped to it because this component needs the client side to work as intended.

3. Next, open the `app/layout.tsx` file and import the newly created `ThemeProvider` component. Wrap the existing content inside the `ThemeProvider` component to enable dark theme support throughout the application.

```
import type { Metadata } from "next";
import { Inter, Space_Grotesk } from "next/font/google";
import "./globals.css";
import { ThemeProvider } from "@/components/ui/theme-provider";

const inter = Inter({
  variable: "--font-inter",
  subsets: ["latin"],
  weight: ["100", "200", "300", "400", "500", "600", "700", "800", "900"],
});

const spaceGrotesk = Space_Grotesk({
  variable: "--font-space-grotesk",
  subsets: ["latin"],
  weight: ["300", "400", "500", "600", "700"],
});

export const metadata: Metadata = {
  title: "Neo LMS",
  description: "A Learning Management System for Modern Education",
  icons: {
    icon: '/favicon-black.png',
  },
};

export default function RootLayout({
  children,
```

```
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html lang="en" suppressHydrationWarning>
      <body
        className={`${inter.variable} ${spaceGrotesk.variable} antialiased`}
      >
        <ThemeProvider attribute="class" defaultTheme="system" enableSystem
disableTransitionOnChange>
          {children}
        </ThemeProvider>
      </body>
    </html>
  );
}
```

4. Add a mode toggle
   Place a mode toggle on your site to toggle between light and dark mode. You can use shadcn's `Toggle` component for this purpose. `app/components/ui/theme-toggle.tsx`.

Here's how to implement it:

```
"use client"

import * as React from "react"
import { Moon, Sun } from "lucide-react"
import { useTheme } from "next-themes"

import { Button } from "@/components/ui/button"
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuTrigger,
} from "@/components/ui/dropdown-menu"

export function ModeToggle() {
  const { setTheme } = useTheme()

  return (
    <DropdownMenu>
      <DropdownMenuTrigger asChild>
        <Button variant="outline" size="icon">
          <Sun className="h-[1.2rem] w-[1.2rem] scale-100 rotate-0 transition-
all dark:scale-0 dark:-rotate-90" />
          <Moon className="absolute h-[1.2rem] w-[1.2rem] scale-0 rotate-90
transition-all dark:scale-100 dark:rotate-0" />
          <span className="sr-only">Toggle theme</span>
```

```
          </Button>
        </DropdownMenuTrigger>
        <DropdownMenuContent align="end">
          <DropdownMenuItem onClick={() => setTheme("light")}>
            Light
          </DropdownMenuItem>
          <DropdownMenuItem onClick={() => setTheme("dark")}>
            Dark
          </DropdownMenuItem>
          <DropdownMenuItem onClick={() => setTheme("system")}>
            System
          </DropdownMenuItem>
        </DropdownMenuContent>
      </DropdownMenu>
    )
  }
```